

Fork()- Create a new process.

Simplest possible fork() program

```
#include <stdio.h>

#include <unistd.h>

int main() {
    fork();

    printf("Hello\n");

    return 0;
}
```

After running this?

It prints Hello **twice**, not once.

Why? Because fork() splits your program into **two processes** at that exact line.

Before fork(), there's one process. **After fork()**, there are two — and **both of them** continue running the rest of the code (printf("Hello\n") and everything after it) **independently**.

Think of it like this: imagine you're reading a recipe out loud, and at one specific line, you get magically cloned into two people. Both clones keep reading the rest of the recipe out loud, at the same time, from that point onward.

- **What is unistd.h?**

unistd.h stands for "**Unix standard**". It's a **header file** —

unistd.h gives you functions for interacting with the **operating system itself** —

It contains declarations for functions that let your C program talk directly to the Unix/Linux operating system, like creating processes.

stdio.h gives you functions for input/output (printf, scanf).

- **Now let's understand the return value**

fork() gives back a number (called pid) —

```
#include <stdio.h>

#include <unistd.h>

int main() {
    int pid;

    pid = fork();

    printf("pid value = %d\n", pid);

    return 0;
}
```

Result: Two lines printed

pid value = 4821

pid value = 0

Explanation

- One of those two processes is the **original one** -- the **parent**. In the parent, pid holds the **child's process ID** (some number like 4821).
- The other process is the **new one** -the **child**. In the child, pid is always exactly **0**.

So the same line `pid = fork();` gives two different answers depending on *which* process is now reading it.

- **PID = Process ID**

A **PID** is a unique number the operating system assigns to every running process.

Using that to make them do different things

Now that we know the child always gets `pid == 0`, we can use a simple if to separate their behavior:

```
#include <stdio.h>
#include <unistd.h>

int main() {
    int pid;

    pid = fork();

    if (pid == 0) {
        printf("I am the child\n");
    } else {
        printf("I am the parent\n");
    }

    return 0;
}
```

Output (order may vary):

I am the child

I am the parent

Header files/ Function

Other common functions from unistd.h

You'll run into these a lot in this lab:

Function	What it does
printf()	to print output
fork()	Creates a new process
getpid()	Returns the current process's ID
getppid()	Returns the parent process's ID
sleep(seconds)	Pauses execution for a given time
execl(), execvp(), etc.	Some exec variants also live here (others are in unistd.h or sys/wait.h depending on which one)

Note : return is a built-in language keyword in C. it is not contained in any header file.

What return 0; actually does

It does two things:

1. Ends the currently executing function (in this case, main()).
2. Sends the value 0 back to whoever called that function.

Header files (the "toolboxes" we need)

- stdio.h — gives us printf() (to print output) and perror() (to print an error message when something fails).
- stdlib.h — gives us exit() (to stop the program immediately) and the constant EXIT_FAILURE (a standard value meaning "the program failed", usually 1).
- unistd.h — gives us fork(), getpid(), getppid() as discussed earlier.
- sys/types.h — gives us the special data type **pid_t**, which is specifically designed to hold process IDs (instead of using a plain int).

value of variable pid (from
fork())

value returned by
getpid()

value returned by
getppid()

In the child	0 (a flag meaning "you're the child")	The child's real PID, e.g. 4823 (X)	The parent's real PID, e.g. 4822 (Y)
In the parent	The child's real PID, e.g. 4823 (X)	The parent's real PID, e.g. 4822 (Y)	The parent's real PID, e.g. 4822 (Y)

Explanation of code

Header files (the "toolboxes" we need)

```
#include <stdio.h>    // For printf() and perror()
#include <stdlib.h>    // For exit() and EXIT_FAILURE
#include <unistd.h>    // For fork(), getpid(), and getppid()
#include <sys/types.h> // For pid_t
```

- `stdio.h` — gives us `printf()` (to print output) and `perror()` (to print an error message when something fails).
- `stdlib.h` — gives us `exit()` (to stop the program immediately) and the constant `EXIT_FAILURE` (a standard value meaning "the program failed", usually 1).
- `unistd.h` — gives us `fork()`, `getpid()`, `getppid()` as discussed earlier.
- `sys/types.h` — gives us the special data type `pid_t`, which is specifically designed to hold process IDs (instead of using a plain `int`).

Step 1: Calling `fork()`

```
pid_t pid = fork();
```

At this exact line, the operating system clones the process. Now there are **two** processes running this same program, both continuing from the line right after this one.

- In the **parent**, `pid` = the child's process ID (some positive number).
- In the **child**, `pid` = 0.
- If the `fork` itself failed (e.g., system ran out of resources), `pid` = a negative number, and only **one** process exists (no child was created).

Step 2: Checking if `fork()` failed

```
if (pid < 0) {
    perror("Fork failed");
    exit(EXIT_FAILURE);
}
```

- `pid < 0` means the OS could not create a new process.

- perror("Fork failed") prints your custom message **plus** the actual system error reason (e.g., Fork failed: Resource temporarily unavailable).
- exit(EXIT_FAILURE) immediately terminates the program with a failure status. This is important — if fork failed, there's no child, so we shouldn't continue as if there were.

Step 3: The child's path

```
} else if (pid == 0) {  
    printf("[CHILD] PID: %d | Parent PID (PPID): %d\n", getpid(), getppid());
```

- This branch runs **only in the child process** (since only the child sees pid == 0).
- getpid() → returns the child's own process ID.
- getppid() → returns the child's parent's process ID (i.e., the PID of the process that called fork()).
- So this line prints something like:
- [CHILD] PID: 4823 | Parent PID (PPID): 4822

Step 4: The parent's path

```
} else {  
    printf("[PARENT] PID: %d | Spawned Child PID: %d\n", getpid(), pid);  
}
```

- This branch runs **only in the parent process** (since here pid is a positive number, not 0 and not negative).
- getpid() → returns the parent's own process ID.
- pid (the variable) → still holds the child's PID, because that's what fork() returned to the parent.
- So this line prints something like:
- [PARENT] PID: 4822 | Spawned Child PID: 4823

Step 5: Returning

```
return 0;
```

- Both the parent and the child reach this line independently and exit normally (0 = success).
- Note: both processes execute this return 0; — it's not shared, each process has its own separate copy of the entire function's execution.

Example full output

[CHILD] PID: 4823 | Parent PID (PPID): 4822

[PARENT] PID: 4822 | Spawned Child PID: 4823

exec() replaces a running program with a brand new program.

Think of it like this: imagine you're an actor on stage playing Character A. Instead of finishing your scene and letting a new actor come out for Character B, you *magically transform into* Character B mid-scene — same stage, same spot, but now you're a completely different character with a different script.

That's what `exec()` does to a running process.

Key points:

- The process keeps its **same PID** (process ID) — it doesn't create a new process.
- But its **code, data, and memory get completely wiped and replaced** by the new program.
- If `exec()` succeeds, it **never returns** — because the old program is gone. There's no "come back here" after this.

Why is it useful?

It's almost always paired with `fork()`:

1. `fork()` — makes a copy of the current process (like cloning yourself).
2. `exec()` — the copy then transforms into a totally different program.

The naming pattern (key to remember)

`exec` + [l or v] + [p] + [e]

Each letter added means one specific thing:

Letter Meaning

- l** **list** — arguments passed individually, one by one
- v** **vector** — arguments passed as an array (`char *argv[]`)
- p** **PATH** — searches PATH env variable; you give just the program name, not full path
- e** **environment** — lets you pass a custom environment (`char *envp[]`) instead of inheriting the current one

Argument breakdown (same pattern as before)

```
execlp(file, arg0, arg1, NULL)
```

```
"ls", "ls", "-l", NULL
```

Arg	Value	Meaning
-----	-------	---------

file	"ls"	Name to search for in PATH
------	------	----------------------------

arg0	"ls"	What the program sees as its own name (argv[0])
------	------	---

arg1	"-l"	Actual option passed to ls (argv[1])
------	------	--------------------------------------

terminator	NULL	Marks end of argument list
------------	------	----------------------------

Includes

c

```
#include <stdio.h>    // printf(), perror()
#include <stdlib.h>    // exit(), EXIT_FAILURE
#include <unistd.h>    // fork(), execlp()
#include <sys/types.h> // pid_t
```

These just bring in the function/type declarations you need.

Step-by-step execution

Step 1: fork()

```
pid_t pid = fork();
```

This creates a **new process** — a near-identical copy of the current one (same code, same variables, same point of execution).

Key thing: after this line, **both processes exist and both continue running from this exact point** — but fork() returns a *different value* to each:

- In the **parent**, pid = the child's actual process ID (a positive number)
- In the **child**, pid = 0

Step 2: if (pid < 0) — fork failed

```
if (pid < 0) {
    perror("Fork failed");
    exit(EXIT_FAILURE);
}
```

If fork() itself failed (e.g., system out of resources), only the original process exists. It prints an error and exits. In a normal run, this branch is skipped.

Step 3: else if (pid == 0) — this is the CHILD

```

} else if (pid == 0) {
    printf("[CHILD] Replacing process image with 'ls -l'...\n");
    // execvp replaces child code. Arguments: (file, arg0, arg1, ..., NULL)
    execvp("ls", "ls", "-l", NULL);
    // Reached ONLY if execvp fails
    perror("execvp failed");
    exit(EXIT_FAILURE);
}

```

This block only runs in the **child** process.

1. It prints [CHILD] Replacing process image with 'ls -l'...
2. `execvp("ls", "ls", "-l", NULL)` runs — this **replaces the child's entire memory image** with the `ls` program (found via `PATH`), and runs it as `ls -l`.
 - If this succeeds: the child process **is now ls**. It runs, lists the current directory's contents in long format, and exits on its own. The lines *after* `execvp()` (the `perror/exit`) are **never reached**, because that code no longer exists in this process once `exec` swaps it out.
 - If it fails (e.g., `ls` isn't found): execution falls through to `perror("execvp failed")` and `exit(EXIT_FAILURE)`.

Step 4: else — this is the PARENT

```

} else {
    printf("[PARENT] Child spawned. Parent continuing execution...\n");
}

```

This block only runs in the **parent** process (where `pid` is the child's PID, a positive number). It just prints a message and keeps going — it does **not** wait for the child to finish.

Step 5: return 0;

Both the parent (after the `else` block) and the child (only if `execvp` *failed*, since it would've already `exit()`ed otherwise) reach the end of `main()`. If `execvp` succeeded in the child, this line is never reached by the child — it's a completely different program by then.

total 24 — what it is

When you run `ls -l`, the **very first line** of output (before the file listing) is always:

total N

It's the **total number of disk blocks** used by all the files in that directory listing, as reported by the filesystem.

Specifically:

- Each "block" is typically **1 KB** (1024 bytes) on most Linux systems (this can vary by filesystem/config).
- So total 24 means: the files shown below use **24 blocks ≈ 24 KB of disk space** (in block-allocation terms, not exact byte size).

`execvp()` is used to **replace the current process with another program**. The basic syntax is:

```
execvp("program", "program", "arg1", "arg2", ..., NULL);
```

The first argument is the program to execute, and the remaining arguments become `argv[]` of that program.

1. ls -l

```
execvp("ls", "ls", "-l", NULL);
```

Equivalent terminal command:

```
ls -l
```

Here:

- "ls" → program to find and execute
- "ls" → `argv[0]`
- "-l" → `argv[1]`
- NULL → marks the end of arguments

2. ls -a

```
execvp("ls", "ls", "-a", NULL);
```

Equivalent:

```
ls -a
```

3. ls -l -a

```
execvp("ls", "ls", "-l", "-a", NULL);
```

Equivalent:

```
ls -l -a
```

4. mkdir mydir

```
execvp("mkdir", "mkdir", "mydir", NULL);
```

Equivalent:

```
mkdir mydir
```

5. mkdir -p test/a/b

```
execlp("mkdir", "mkdir", "-p", "test/a/b", NULL);
```

Equivalent:

```
mkdir -p test/a/b
```

6. cat file.txt

```
execlp("cat", "cat", "file.txt", NULL);
```

Equivalent:

```
cat file.txt
```

7. cp file1.txt file2.txt

```
execlp("cp", "cp", "file1.txt", "file2.txt", NULL);
```

Equivalent:

```
cp file1.txt file2.txt
```

8. rm file.txt

```
execlp("rm", "rm", "file.txt", NULL);
```

Equivalent:

```
rm file.txt
```

wait(): the parent process **pauses until the child process is executed**, then parent process reads the status **returned by child process**.

Example code 1

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();

    if (pid == 0) {
        // Child
        printf("Child: working...\n");
        exit(5);
    } else {
```

```

    // Parent

    int status;

    wait(&status); // pause until child process executes

    printf("Parent: child execution done, result = %d\n", WEXITSTATUS(status));
}

return 0;
}

```

Working

1. `fork()` creates the child process.
2. **Child** prints a message and exits with code 5.
3. **Parent** calls `wait(&status)` and pauses until the child process finishes.
4. Parent reads the result with `WEXITSTATUS(status)` and prints it.

Output

Child: working...

Parent: child done, result = 5

Example code 2

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
        exit(1);
    }
    else if (pid == 0) {
        // Child: pretend to do a task
        int number = 7;
        printf("Child: checking if %d is even...\n", number);

        if (number % 2 == 0)
            exit(0); // success
        else
            exit(1); // failure
    }
    else {
        // Parent

```

```
int status;
wait(&status);

if (WIFEXITED(status)) {
    int code = WEXITSTATUS(status);

    if (code == 0)
        printf("Parent: child succeeded (number is even)\n");
    else
        printf("Parent: child failed (number is odd), code = %d\n", code);
}
}

return 0;
}
```

Explanation of code

Step 1: Include the headers

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
```

Step 2: Create the child

```
pid_t pid = fork();
```

fork() makes a copy of the running program. Now **two processes** run the code below this line. Each one gets a different pid:

Step 3: Check if fork() failed

```
if (pid < 0) {
    perror("fork failed");
    exit(1);
}
```

If pid is negative, no child was created. Print an error and quit. This rarely happens.

Step 4: Child's code (pid == 0)

```
else if (pid == 0) {
```

```
int number = 7;

printf("Child: checking if %d is even...\n", number);

if (number % 2 == 0)
    exit(0);    // success
else
    exit(1);    // failure
}
```

Only the child runs this block:

1. Sets number = 7.
2. Prints a message.
3. Checks 7 % 2. The remainder is 1, not 0, so 7 is **odd**.
4. The else runs, so the child calls exit(1).

exit(1) ends the child and leaves the result **1** (failure) for the parent.

Step 5: Parent's code (pid > 0)

```
else {
    int status;
    wait(&status);
}
```

Only the parent runs this block:

1. status is an empty box for the child's result.
 2. wait(&status) **pauses the parent** until the child finishes, then fills status with the result.
-

Step 6: Parent reads the result

```
if (WIFEXITED(status)) {
    int code = WEXITSTATUS(status);
}
```

- WIFEXITED(status) asks: "Did the child end normally?" Yes, it called exit(1).
 - WEXITSTATUS(status) extracts the exit code. Here, code = 1.
-

Step 7: Parent decides what to print

```
if (code == 0)
    printf("Parent: child succeeded (number is even)\n");
else
    printf("Parent: child failed (number is odd)\n");
```

else

```
printf("Parent: child failed (number is odd), code = %d\n", code);
```

- code == 0 means success.
- code is 1, so the else runs.

Step 8: End

```
return 0;
```

The parent finishes cleanly.

Output

Child: checking if 7 is even...

Parent: child failed (number is odd), code = 1

Step-by-Step Explanation

Step 1: Headers

C

```
#include <stdio.h>    // printf, perror
```

```
#include <stdlib.h>    // exit, EXIT_FAILURE
```

```
#include <unistd.h>    // fork, sleep
```

```
#include <sys/types.h> // pid_t
```

```
#include <sys/wait.h>  // wait, WIFEXITED, WEXITSTATUS
```

These give you the functions the program uses.

Step 2: Declare status and call fork()

C

```
int status;
```

```
pid_t pid = fork();
```

- status is an empty box that will later hold the child's result.
- fork() creates a copy of the program. Now **two processes** run the code below.

Process Value of pid

Child 0

Parent child's real PID (e.g., 4821)

Failure -1

Step 3: Check for failure

c

```
if (pid < 0) {  
    perror("Fork failed");  
    exit(EXIT_FAILURE);  
}
```

If fork() failed, print an error and quit.

Step 4: Child's code (pid == 0)

c

```
else if (pid == 0) {  
    printf("[CHILD] Working... sleeping for 2 seconds.\n");  
    sleep(2);  
    printf("[CHILD] Exiting with status code 42.\n");  
    exit(42);  
}
```

Only the child runs this:

1. Prints that it is working.
 2. sleep(2) pretends to do work for 2 seconds.
 3. Prints that it is exiting.
 4. exit(42) ends the child and leaves the result **42** for the parent.
-

Step 5: Parent prints and waits

```
c
else {
    printf("[PARENT] Waiting for child (PID: %d) to complete...\n", pid);
    pid_t terminated_child = wait(&status);
```

Only the parent runs this:

1. Prints a message with the child's PID.
2. `wait(&status)` **pauses the parent** for about 2 seconds, until the child exits.
3. When the child ends, `wait()` returns the **child's PID** (saved in `terminated_child`) and fills `status` with the result. The child is now **reaped**.

Step 6: Parent reads the result

```
c
if (WIFEXITED(status)) {
    printf("[PARENT] Child PID %d finished normally with exit code: %d\n",
        terminated_child, WEXITSTATUS(status));
}
```

- `WIFEXITED(status)` asks: "Did the child end normally?" Yes, it called `exit(42)`.
- `WEXITSTATUS(status)` extracts the exit code: **42**.
- The parent prints the child's PID and the code.

Step 7: End

```
c
return 0;
```

The parent finishes cleanly.

Timeline

Time	Parent	Child
0s	<code>fork()</code> ----->	created
0s	prints "Waiting..."	prints "Working..."
0s	<code>wait()</code> -> sleeping...	<code>sleep(2)</code> ...
2s		prints "Exiting..."
2s		<code>exit(42)</code>
2s	wakes up, <code>status</code> holds 42	

2s prints "finished, code: 42"

Output

[PARENT] Waiting for child (PID: 4821) to complete...

[CHILD] Working... sleeping for 2 seconds.

[CHILD] Exiting with status code 42.

[PARENT] Child PID 4821 finished normally with exit code: 42